求和

内容

等比数列求和及其扩展

数论分块

等比数列的和

问题

给定整数 A，正整数 N, M，求 $1 + A + A^2 + \cdots + A^{N-1}$，模 M。

定义

对于非负整数 n，考虑多项式 $f_n(x) := 1 + x + x^2 + \cdots + x^{n-1}$。
特别的，$f_0(x) = 0$。

我们要求的就是 $f_N(A) \bmod M$。

- $f_N(A) = \frac{A^N - 1}{A - 1}$，如果 $(A-1)$ 和 M 互素，那么 $(A-1)$ 在模 M 下有乘法逆元。
- 以下我们考虑一般情况，即 $(A-1)$ 和 M 未必互素。

$$f_n(x) = 1 + x + x^2 + \cdots + x^{n-1}$$

当 n 是偶数时，我们把偶次项和奇次项分开，写成

$$f_n(x) = (1 + x^2 + x^4 + \cdots + x^{n-2}) + (x + x^3 + x^5 + \cdots + x^{n-1})$$

注意到

$$1+x^2+x^4+\cdots+x^{n-2}=1+x^2+(x^2)^2+\cdots+(x^2)^{n/2-1}=f_{n/2}(x^2)$$

而

$$x+x^3+x^5+\cdots+x^{n-1}=x\cdot(1+x^2+x^4+\cdots+x^{n-2})=x\cdot f_{n/2}(x^2)$$

于是有

$$\boxed{f_n(x)=f_{n/2}(x^2)+x\cdot f_{n/2}(x^2),\quad n\text{ 是偶数}.}$$

当 n 是奇数时，

$$f_n(x)=1+x\cdot(1+x+\cdots+x^{n-2})=1+x\cdot f_{n-1}(x).$$

命题

$f_0(x) = 0$。对于 $n \geq 1$，有

$$f_n(x) = \begin{cases} (1+x) f_{n/2}(x^2), & n \text{ 是偶数}, \\ 1 + x \cdot f_{n-1}(x), & n \text{ 是奇数}. \end{cases}$$

- 这跟计算 x^n 的快速幂算法有点像。

例题 Geometric Progression

abc293_e

给定整数 A, X, M，求 $\sum_{i=0}^{X-1} A^i$，模 M。

- $1 \le A, M \le 10^9$
- $1 \le X \le 10^{12}$

递归写法

```
int mod;

int power_sum(long long x, long long n) {
    if (n == 0)
        return 0;

    if (n & 1)
        return (1 + x * power_sum(x, n - 1)) % mod;

    return (1 + x) * power_sum(x * x % mod, n >> 1) % mod;
}
```

非递归写法

```
int power_sum(long long x, long long n, int mod) {
    long long ans = 0;
    long long prod = 1;
    // 答案 == ans + prod * power_sum(x, n)
    while (n) {
        if (n & 1) {
            ans = (ans + prod) % mod;
            prod = (prod * x) % mod;
        }
        prod = prod * (1 + x) % mod;
        x = x * x % mod;
        n >>= 1;
    }
    return ans;
}
```

习题 LCM 111

arc050_c

整数 x 的十进制写法是 A 个 1，整数 y 的十进制写法是 B 个 1。求 x 和 y 的最小公倍数除以 M 的余数。

- $1 \leq A, B \leq 10^{18}$
- $2 \leq M \leq 10^{9}$

分析

我们知道 $\mathrm{lcm}(x,y)=\frac{xy}{\gcd(x,y)}$，考虑 $\gcd(x,y)$。

我们把十进制写法是 n 个 1 的整数记作 $f(n)$，那么 $x=f(A),y=f(B)$。

注意到

$$\gcd(f(A),f(B))=f(\gcd(A,B)).$$

设 $d=\gcd(A,B)$，又设 $A=kd$。

注意到

$$f(kd)/f(d)=1+10^d+(10^d)^2+\cdots+(10^d)^{k-1}.$$

所以问题就归结为计算两个等比数列的和。

```
int power(int x, long long n, int mod) {
    int ans = 1;
    while (n > 0) {
        if (n & 1)
            ans = (long long) ans * x % mod;
        x = (long long) x * x % mod;
        n >>= 1;
    }
    return ans;
}

int power_sum(int x, long long n, int mod) {
   int ans = 0;
   int prod = 1;
   while (n > 0) {
        if (n & 1) {
            ans = (ans + prod) % mod;
            prod = (long long) prod * x % mod;
        }
        prod = (long long) prod * (1 + x) % mod;
        x = (long long) x * x % mod;
        n >>= 1;
   }
   return ans;
}

int main() {
    long long a, b;
    int m;
    cin >> a >> b >> m;
    long long d = __gcd(a, b);
    int ans = (long long) power_sum(power(10, d, m), a / d, m) * power_sum(10, b, m) % m;
    cout << ans << '\n';
}
```

扩展

问题

给定非负整数 N, K，整数 A 和正整数 M，求 $\sum_{i=0}^{N-1} i^K A^i$，模 M。

对于非负整数 n, k，考虑多项式 $f_n^k(x) = \sum_{i=0}^{n-1} i^k x^i$。

我们要求的就是 $f_N^K(A) \bmod M$。

$$f_n^k(x) = \sum_{i=0}^{n-1} i^k x^i.$$

当 n 是偶数时，我们把偶次项和奇次项分开，写成

$$f_n^k(x) = \sum_{i=0}^{n/2-1} (2i)^k x^{2i} + \sum_{i=0}^{n/2-1} (2i+1)^k x^{2i+1}$$

偶次项

$$\sum_{i=0}^{n/2-1}(2i)^k x^{2i}$$

$$= 2^k \cdot \sum_{i=0}^{n/2-1} i^k (x^2)^i$$

$$= 2^k \cdot f_{n/2}^k(x^2)$$

奇次项

$$\sum_{i=0}^{n/2-1}(2i+1)^k x^{2i+1}$$

$$= x \sum_{i=0}^{n/2-1}(2i+1)^k x^{2i}$$

$$= x \sum_{i=0}^{n/2-1} \sum_{t=0}^{k} \binom{k}{t} (2i)^t x^{2i}$$

$$= x \sum_{t=0}^{k} \binom{k}{t} 2^t \sum_{i=0}^{n/2-1} i^t (x^2)^i$$

$$= x \sum_{t=0}^{k} \binom{k}{t} 2^t \cdot f_{n/2}^t(x^2)$$

当 n 是偶数时，有

$$
\begin{aligned}
f_n^k(x) &= 2^k \cdot f_{n/2}^k(x^2) + x \sum_{t=0}^{k} \binom{k}{t} 2^t \cdot f_{n/2}^t(x^2) \\
&= 2^k(1+x) f_{n/2}^k(x^2) + x \sum_{t=0}^{k-1} \binom{k}{t} 2^t \cdot f_{n/2}^t(x^2)
\end{aligned}
$$

$$f_n^k(x) = \sum_{i=0}^{n-1} i^k x^i.$$

当 n 是奇数时

$$\begin{aligned}
f_n^k(x) &= 0^k + \sum_{i=1}^{n-1} i^k x^i \\
&= 0^k + \sum_{i=0}^{n-2} (i+1)^k x^{i+1} \\
&= 0^k + x \sum_{i=0}^{n-2} (i+1)^k x^i
\end{aligned}$$

$$\begin{aligned}
&= 0^k + x \sum_{i=0}^{n-2} \sum_{t=0}^{k} \binom{k}{t} i^t x^i \\
&= 0^k + x \sum_{t=0}^{k} \binom{k}{t} \sum_{i=0}^{n-2} i^t x^i \\
&= 0^k + x \sum_{t=0}^{k} \binom{k}{t} f_{n-1}^t(x)
\end{aligned}$$

命题

$f_0^k(x) = 0$。

对于 $n \geq 1$，有

$$f_n^k(x) = \begin{cases} 2^k(1+x)f_{n/2}^k(x^2) + x\sum_{t=0}^{k-1}\binom{k}{t}2^t \cdot f_{n/2}^t(x^2), & n \text{ 是偶数} \\ 0^k + x\sum_{t=0}^{k}\binom{k}{t}f_{n-1}^t(x), & n \text{ 是奇数} \end{cases}$$

$$f_n^k(x) = \begin{cases} 2^k(1+x)f_{n/2}^k(x^2) + x\sum_{t=0}^{k-1}\binom{k}{t}2^t \cdot f_{n/2}^t(x^2), & n \text{ 是偶数} \\ 0^k + x\sum_{t=0}^{k}\binom{k}{t}f_{n-1}^t(x), & n \text{ 是奇数} \end{cases}$$

我们可以把上面的式子看作一个**递推式**：

- 若 n 是偶数，我们由 $f_{n/2}^0(x^2), f_{n/2}^1(x^2), \ldots, f_{n/2}^k(x^2)$ 推出 $f_n^k(x)$。
- 若 n 是奇数，我们由 $f_{n-1}^0(x), f_{n-1}^1(x), \ldots, f_{n-1}^k(x)$ 推出 $f_n^k(x)$。

更进一步，我们也可以说

- 若 n 是偶数，我们由 $f_{n/2}^0(x^2), f_{n/2}^1(x^2), \ldots, f_{n/2}^k(x^2)$ 推出 $f_n^0(x), f_n^1(x), \ldots, f_n^k(x)$。
- 若 n 是奇数，我们由 $f_{n-1}^0(x), f_{n-1}^1(x), \ldots, f_{n-1}^k(x)$ 推出 $f_n^0(x), f_n^1(x), \ldots, f_n^k(x)$。

总之，利用上面的式子，我们可以在 $O(k^2)$ 的时间内，由一个长为 $k+1$ 的序列推出另一个长为 $k+1$ 的序列。

例题 数列求和

洛谷P4948

给定整数 n, a, k，求 $\sum_{i=1}^{n} i^k a^i$，模 $10^9 + 7$。

- $1 \le n \le 10^{18}$
- $1 \le a \le 10^9$
- $0 \le k \le 2000$

递归写法

```
const int mod = 1e9 + 7;
const int maxk = 2005;
int C[maxk][maxk]; // 组合数
int p2[maxk]; // p2[i]：2的i次方
int k;

vector<long long> solve(int x, long long n) { //返回一个长度是 k + 1 的数组
    vector<long long> ans(k + 1);
    if (n == 0)
        return ans;
    if (n & 1) {
        auto f = solve(x, n - 1);
        for (int i = 0; i <= k; i++) {
            for (int j = 0; j <= i; j++)
                ans[i] += C[i][j] * f[j] % mod;
            ans[i] = ans[i] % mod * x;
        }
        ans[0]++;
        for (int i = 0; i <= k; i++)
            ans[i] %= mod;
        return ans;
    }
    auto f = solve((long long) x * x % mod, n / 2);
    for (int i = 0; i <= k; i++) {
        for (int j = 0; j < i; j++)
            ans[i] += f[j] * C[i][j] % mod * p2[j] % mod;
        ans[i] = (ans[i] % mod * x + (f[i] * p2[i] % mod * (1 + x) % mod)) % mod;
    }
    return ans;
}
```

```
int main() {
    long long n;
    int a;
    cin >> n >> a >> k;
    for (int i = 0; i <= k; i++) {
        C[i][0] = 1;
        for (int j = 1; j <= i; j++)
            C[i][j] = (C[i - 1][j] + C[i - 1][j - 1]) % mod;
    }

    p2[0] = 1;
    for (int i = 1; i <= k; i++)
        p2[i] = p2[i - 1] * 2 % mod;

    long long ans = solve(a, n + 1)[k];

    if (k == 0) {
        ans--;
        if (ans < 0)
            ans += mod;
    }
    cout << ans << '\n';
}
```

非递归写法

仿照前面的等比数列求和的非递归写法，我们维护一列系数 $w_0, w_1, \ldots, w_K$ 以及三个变量 s、x、n，使得等式

$$f_N^K(A) = s + \sum_{i=0}^{K} w_i \cdot f_n^i(x)$$

总是成立。

一开始 $s = 0$，$x = A$，$n = N$，$w_K = 1$，$w_0, w_1, \ldots, w_{K-1}$ 都是 0。

最后，n 变成 0 而 s 就是所求的 $f_N^K(A)$。

当 n 是奇数时，我们有

$$
\begin{aligned}
s+\sum_{i=0}^{K} w_i \cdot f_n^i(x) &= s+\sum_{i=0}^{K} w_i \cdot \left(0^i + x \sum_{j=0}^{i} \binom{i}{j} f_{n-1}^j(x)\right) \\
&= s + w_0 + \sum_{i=0}^{K} w_i \cdot \left(x \sum_{j=0}^{i} \binom{i}{j} f_{n-1}^j(x)\right) \\
&= s + w_0 + x \sum_{i=0}^{K} w_i \sum_{j=0}^{i} \binom{i}{j} f_{n-1}^j(x) \\
&= s + w_0 + x \sum_{j=0}^{K} f_{n-1}^j \sum_{i=j}^{K} w_i \binom{i}{j}
\end{aligned}
$$

$n \leftarrow n-1$；x 不变；$s \leftarrow s + w_0$；$w_j \leftarrow x \sum_{i=j}^{K} w_i \binom{i}{j}$。

当 n 是偶数时，我们有

$$
\begin{aligned}
s + \sum_{i=0}^{K} w_i \cdot f_n^i(x) &= s + \sum_{i=0}^{K} w_i \cdot (2^i \cdot f_{n/2}^i(x^2) + x \sum_{j=0}^{i} \binom{i}{j} 2^j \cdot f_{n/2}^j(x^2)) \\
&= s + \sum_{j=0}^{K} f_{n/2}^j(x^2) \cdot 2^j (w_j + x \sum_{i=j}^{K} \binom{i}{j} w_i)
\end{aligned}
$$

$n \leftarrow n/2$；s 不变；$w_j \leftarrow 2^j(w_j + x \sum_{i=j}^{K} \binom{i}{j} w_i)$；$x \leftarrow x^2$。

```cpp
const int mod = 1e9 + 7;
const int maxk = 2005;
int C[maxk][maxk]; // 组合数
int p2[maxk]; // 二的幂

int solve(int x, long long n, int k) {
    vector<long long> w(k + 1);
    w[k] = 1;
    long long s = 0;
    while (n > 0) {
        if (n & 1) {
            s += w[0];
            for (int i = 0; i <= k; i++) {
                for (int j = i + 1; j <= k; j++)
                    w[i] += w[j] * C[j][i] % mod;
                w[i] = w[i] % mod * x % mod;
            }
        }
        for (int i = 0; i <= k; i++) {
            long long t = 0;
            for (int j = i; j <= k; j++)
                t += C[j][i] * w[j] % mod;
            w[i] = (w[i] + t % mod * x) % mod * p2[i] % mod;
        }
        x = (long long) x * x % mod;
        n >>= 1;
    }
    return s % mod;
}

int main() {
    long long n;
    int a, k;
    cin >> n >> a >> k;

    // 计算组合数和2的幂

    int ans = solve(a, n + 1, k);
    if (k == 0) {
        ans--;
        if (ans < 0) ans += mod;
    }
    cout << ans << '\n';
}
```

例题 数学作业

对于正整数 n，定义 Concatenate(n) 为，把 $1, 2, \ldots, n$ 依次连接起来得到的数。

例如 Concatenate(13) = 12345678910111213。

给定正整数 N, M，求 Concatenate(N) $\mod M$。

- $1 \le N \le 10^{18}$，$1 \le M \le 10^9$。

对于正整数 l, r（$l \le r$），我们把依次连接 $l, l+1, \ldots, r$ 所得的数记作 $f(l, r)$。那么 Concatenate(n) 就是 $f(1, n)$。

如果 l, r 在十进制下都是 k 位数，那么 $f(l, r)$ 可表为

$$\sum_{i=l}^{r} i \cdot 10^{k(r-i)}.$$

做指标变换，令 $j = r - i$，把上式写成

$$\sum_{j=0}^{r-l} (r-j) \cdot 10^{kj} = r \sum_{j=0}^{r-l} (10^k)^j - \sum_{j=0}^{r-l} j \cdot (10^k)^j.$$

模板

```
int k_power_sum(int x, long long n, int k, int mod) {
    vector<vector<int>> C(k + 1, vector<int>(k + 1));
    for (int i = 0; i <= k; i++) {
        C[i][0] = 1;
        for (int j = 1; j <= i; j++)
            C[i][j] = (C[i - 1][j] + C[i - 1][j - 1]) % mod;
    }
    vector<int> p2(k + 1);
    p2[0] = 1;
    for (int i = 1; i <= k; i++)
        p2[i] = p2[i - 1] * 2 % mod;

    vector<long long> w(k + 1);
    w[k] = 1;
    long long s = 0;
    // 循环不变量：答案 == ans + \sum_{i=0..k} w[i] * f_{n,k}(x)
    while (n > 0) {
        if (n & 1) {
            s += w[0];
            for (int i = 0; i <= k; i++) {
                for (int j = i + 1; j <= k; j++)
                    w[i] += w[j] * C[j][i] % mod;
                w[i] = w[i] % mod * x % mod;
            }
        }
        for (int i = 0; i <= k; i++) {
            long long t = 0;
            for (int j = i; j <= k; j++)
                t += C[j][i] * w[j] % mod;
            w[i] = (w[i] + t % mod * x) % mod * p2[i] % mod;
        }
        x = (long long) x * x % mod;
        n >>= 1;
    }
    return s % mod;
}
```

```
int power(long long x, unsigned long long n, int mod) {
    long long ans = 1;
    while (n > 0) {
        if (n & 1)
            ans = ans * x % mod;
        x = x * x % mod;
        n >>= 1;
    }
    return (int) ans;
}

int main() {
    unsigned long long n;
    int m;
    cin >> n >> m;
    n++;
    long long ans = 0;
    int len = 1;
    for (unsigned long long l = 1; l < n; l *= 10) {
        unsigned long long r = min(l * 10, n);
        long long t = (r - 1) % m * k_power_sum(l * 10 % m, r - l, 0, m) % m - k_power_sum(l * 10 % m, r - l, 1, m);
        ans = ans * power(10, len * (r - l), m) % m + t;
        len++;
    }
    ans %= m;
    if (ans < 0)
        ans += m;
    cout << ans << '\n';
}
```

数论分块

问题

计算 $\sum_{i=1}^{N} f(\lfloor N/i \rfloor)$.

```
long long ans = 0;
for (int i = 1; i <= N; ) {
    int j = N / i;
    int ni = N / j + 1;
    ans += (ni - i) * f(j);
    i = ni;
}
```

问题

计算 $\sum_{i=1}^{N} i \cdot f(\lfloor N/i \rfloor)$.

```
long long ans = 0;
for (int i = 1; i <= N; ) {
    int j = N / i;
    int ni = N / j + 1;
    ans += (i + ni - 1) * (ni - i) / 2 * f(j);
    i = ni;
}
```

例题 余数求和

洛谷P2261

求 $\sum_{i=1}^{n} k \bmod i$.

- $1 \le n, k \le 10^9$.

分析

注意到

$$k \bmod i = k - i \cdot \lfloor k/i \rfloor,$$

于是

$$\sum_{i=1}^{n} k \bmod i = \sum_{i=1}^{n} k - i \cdot \lfloor k/i \rfloor = n(n+1)/2 - \sum_{i=1}^{n} i \cdot \lfloor k/i \rfloor.$$

扩展

问题

给定二元函数 f，计算 $\sum_{i=1}^{n} f(\lfloor n/i \rfloor, \lfloor m/i \rfloor)$。